CDNs Explained

edge PoPs, cache keys, origin shield and purges

Watch the 5-min video

In one line: a CDN serves your content from edge servers near users, so handshakes and round trips are short and cache hits never reach the origin; after a deploy, change file names instead of purging.

1 The problem: content far from users

Your store's servers are in Virginia. A shopper in Mumbai loads the home page: forty product images and three JavaScript bundles, each crossing the world at around **200 ms per round trip**. The page crawls. A **content delivery network** (CDN) fixes this by bringing your content close to the user.

A CDN is a set of edge servers, called **points of presence** (PoPs), spread across the world. Your own servers are the **origin**. Users connect to the nearest PoP, often found through **anycast** or **DNS**. If that PoP has the file cached, it serves it right there, in a few milliseconds.

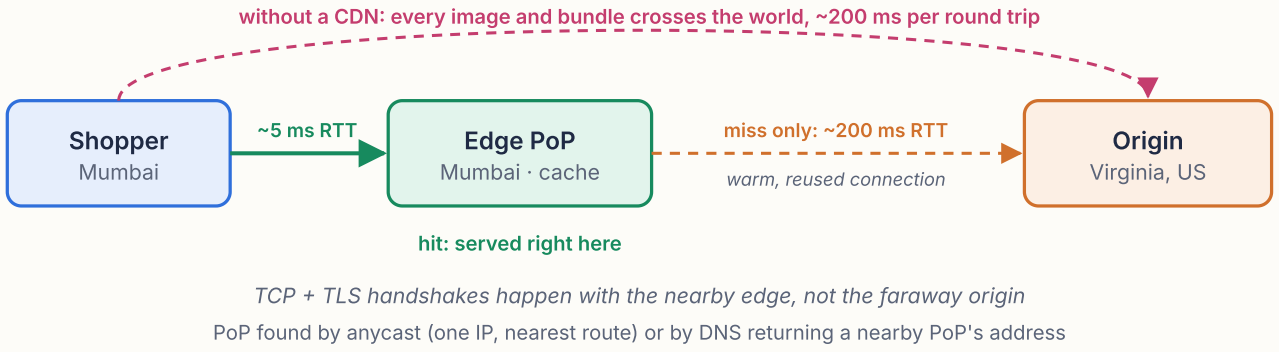


Figure 1. The edge sits a few milliseconds from the user. Only cache misses travel to the origin, over connections the CDN keeps open and warm.

2 How a CDN reduces latency

- **Shorter distance.** The TCP and TLS handshakes happen with a nearby edge, not the faraway origin.
- **Cache hits skip the long trip entirely.**
- **Even misses are faster.** The edge reuses warm, tuned connections back to the origin, so it skips new handshakes.
- **Bonus: offload.** The origin serves far less traffic and survives spikes.

WORKED EXAMPLE

A new HTTPS connection needs about **2 round trips** (TCP, then TLS 1.3) before the first request, and 1 more to get the response. From Mumbai to Virginia: 3 × 200 ms = **600 ms** before the first byte. To a Mumbai PoP at 5 ms: 3 × 5 = **15 ms**, plus a few ms to read from cache. At a **95% hit ratio**, 1 million requests an hour become only **50,000** at the origin (1 in 20).

3 Hit ratio, pull vs push, cache keys and shielding

The key metric is the **cache hit ratio**: the share of requests served from the edge. Most CDNs are **pull CDNs**: on a miss, the edge fetches the file from the origin, caches it and serves later requests itself. **Push CDNs** work the other way: you upload content ahead of time, which suits large, rarely changing files such as video libraries or software downloads.

Each cached object is stored under a **cache key**: usually the URL, plus any headers, cookies or query parameters you choose to include. Include too much, like a session cookie, and every user gets their own copy, so the hit ratio collapses. **Origin shielding** adds a middle cache layer: edge misses go to one shield PoP first, so the origin isn't hit by hundreds of edges at once.

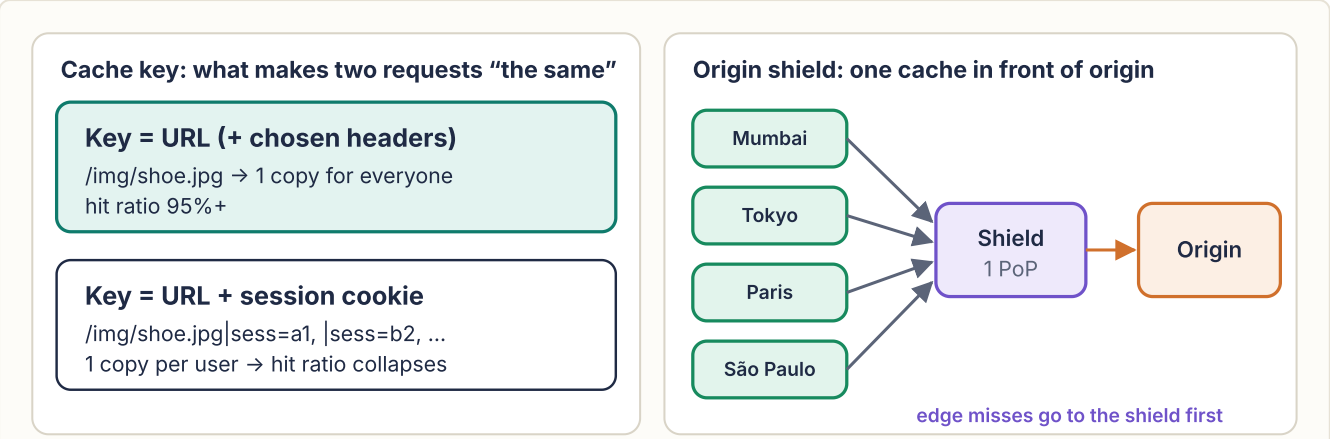


Figure 2. Left: keep the cache key small, or one image becomes thousands of copies. Right: with a shield, a new file costs the origin one request, not one per PoP.

4 Example: images and bundles on CloudFront or Azure Front Door

The build outputs files with a **content hash** in the name, like `app.3f9a.js`. They are served with `Cache-Control: public, max-age=31536000, immutable` (one year, and the browser need not even revalidate). The HTML page itself gets a **short TTL**, such as `max-age=60`, and after each deploy simply points to the new file names.

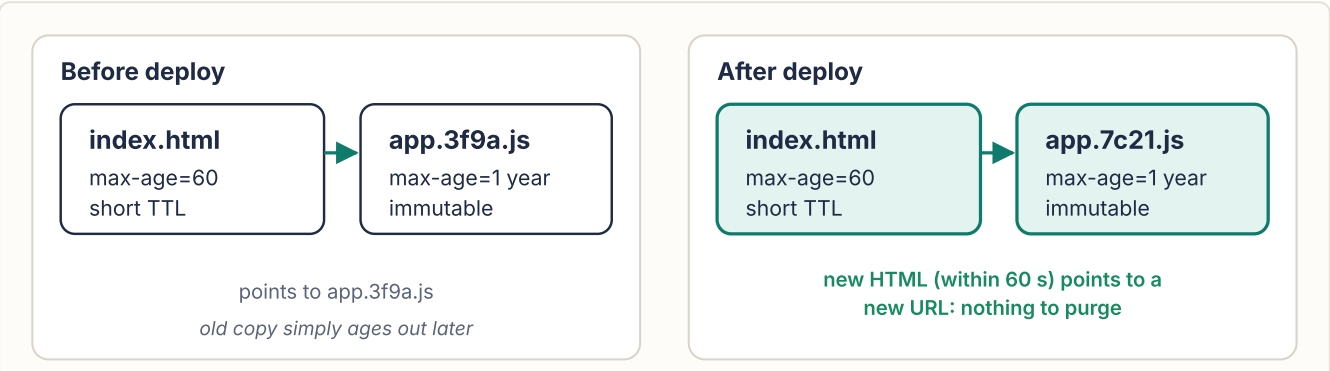


Figure 3. The content hash changes when the file changes, so a new version is a new URL. Only the short-lived HTML has to update.

5 Invalidating after a deploy

The best answer is: **you don't**. Versioned file names mean new content gets a new URL, and old copies simply age out. For things that must keep the same URL, like the HTML or an API response, use **short TTLs**, or issue a **purge**: an invalidation request for specific paths. Purges take time to spread to every PoP (seconds to minutes), and some CDNs charge per path (CloudFront, beyond a monthly free allowance), so they are the exception, not the plan.

COMMON MISTAKES

- Long TTLs on `app.js` without a hash in the name: users keep old code for days.
- Putting cookies or tracking query strings (`?utm_source=…`) in the cache key.
- Caching personalised API responses as `public`: one user's data shown to another.
- Relying on a purge in every deploy script.

6 Key terms

PoP: an edge location with cache servers

Cache hit ratio: share of requests served at the edge

Origin shield: middle cache in front of the origin

Origin: your own servers, the source of truth

Cache key: what identifies one cached object

Purge: invalidate specific cached paths

7 Interview questions

Q1. How does a CDN reduce latency? BASIC

It serves content from edge PoPs near users: handshakes and round trips are short (5 ms instead of 200 ms), cache hits skip the origin entirely, and misses reuse warm edge-to-origin connections.

Q2. Pull CDN vs push CDN? BASIC

Pull: the edge fetches from the origin on the first miss and caches it; easy, and the default for websites. Push: you upload content to the CDN ahead of time; good for large, rarely changing files like video libraries and downloads.

Q3. What is the cache hit ratio, and why does it matter? BASIC

The share of requests served from the edge. At 95%, the origin sees 1 request in 20; at 80%, 1 in 5, which is 4× more origin load. It drives both latency and origin cost.

Q4. How do you invalidate CDN content after a deploy? SCENARIO

Prefer versioned (content-hashed) file names with `max-age=31536000, immutable`, and keep the HTML short-lived (e.g. 60 s) so it points to new names. Purge only the few paths that must change in place.

Q5. Your hit ratio dropped from 95% to 30% after a marketing launch. What do you check? SCENARIO

The cache key. Likely new query parameters (`utm_*`) or a cookie are now part of it, so each visitor creates a new copy. Strip or ignore them in the key, and check `Vary` headers.

Q6. A new 50 MB video goes live and 300 PoPs miss at once. What protects the origin? SCENARIO

Origin shielding: all edge misses go to one shield PoP, which fetches the file once. The CDN also collapses concurrent misses for the same object into one origin request.

Q7. Can you put an API behind a CDN? TRICKY

Yes. Even uncached, it gains nearby TLS termination and warm connections. Cache only public, non-personalised GET responses with short TTLs; send `private` or `no-store` for per-user data.

Q8. Why is the HTML not given the same one-year TTL as the bundles? TRICKY

The HTML's URL never changes, so a long TTL would pin users to the old page, which points to old bundles. It is the entry point that must update, so it gets a short TTL or revalidation.

Next: SD-N-029 · Connection pooling and keep-alive backendroadmap.com